

CMP514 – Computer Graphics and Image Processing Laboratory

B Tech CSE TY

Activity 1 – C Graphics (graphics.h) Lab Activity: Drawing Simple Shapes

Objectives: Introduction to Basic Graphics using graphics.h

Sub activities:

1. Understand the basic setup of a graphics.h program.
2. Learn to initialize and close the graphics mode.
3. Draw fundamental 2D primitives: lines, circles, rectangles.
4. Change drawing colors.
5. Create simple text on the graphics screen.
6. Implement a basic delay for simple animation concepts.

Important Notes:

1. **This is a guided activity.** Follow the instructions carefully.
2. **Don't create a new project from scratch.** A base project is provided to ensure your environment is set up correctly.
3. **Save your work often!** (Usually F2 in Turbo C/C++).
4. **Preferably use DOSBox with Turbo C/C++ compiler to observe graphics output.**
It is most likely to be installed on E drive of computers in lab 303 or you can download and install. Good idea is to use your own laptop.
5. **Pay attention to the comments in the code.** They provide important hints.

Note:

1. In graphics programming, the top left corner of the screen is the origin of coordinate system i.e. $(x,y) = (0,0)$
2. The horizontal axis (X – Axis) grows from left to right and vertical axis (Y – Axis) grows from top to bottom.

Example base_program.c

```
#include <graphics.h> // Include the graphics library
#include <conio.h>      // For getch() or kbhit()
#include <stdio.h>      // For printf (optional)

int main()
{
    int gd = DETECT, gm; // gd = graphics driver, gm = graphics
mode

    // Initialize graphics mode
    initgraph(&gd, &gm, "C:\\TC\\BGI"); // For Turbo C/DOSBox

    // Check if graphics initialization was successful
    if (graphresult() != grOk) {
        printf("Graphics initialization error: %s\n",
grapherrormsg(graphresult()));
        getch();
        return 1;
    }

    // --- Your drawing code will go here ---

    // Example: Draw a simple line to show it's working
    // line(100, 100, 200, 200);

    // Keep the graphics window open until a key is pressed
    printf("Press any key to close the graphics window...\n");
// This will appear in console/text mode
    getch(); // Wait for a key press

    // Close the graphics mode
    closegraph();

    return 0;
}
```

Crucially, the `initgraph` path needs to be correct for the environment. If using DOSBox, ensure `C:\\TC\\BGI` or similar points to the BGI driver files inside the DOSBox mapping.

Activity Sheet:

Task 0: Environment Setup and Initial Run (15 minutes)

1. Start your graphics.h environment:

- If using **DOSBox**: Locate the DOSBox icon/shortcut. Double-click it. This should open a command prompt within DOSBox.
 - Type `cd\` (Enter) to go to the root.
 - Type `cd TC\BIN`
 - Type `TC` (Enter) to start Turbo C/C++ IDE.

2. Open the Base Program:

- In Turbo C/C++: Go to File > Open and navigate to `C:\<YOUR_PROJECT_FOLDER>\base_program.c`

3. Compile and Run:

- In Turbo C/C++: Press `Ctrl+F9` (or Compile > Run).
- You should see a separate graphics window appear (often black). It might show a simple line, or just be blank, and then close when you press a key.

Task 1: Drawing Lines and Rectangles (30 minutes)

You will now add drawing commands within the `main()` function, between `initgraph()` and `getch()`.

1. Open `base_program.c`.
2. Locate the "Your drawing code will go here" section.
3. Draw a Horizontal Line:
 - Add the following line of code:

// Task 1.1: Draw a horizontal red line

```
setcolor(RED); // Set drawing color to Red
```

```
line(50, 100, 250, 100); // line(x1, y1, x2, y2)
```

- Save (`F2` in TC). Compile and Run (`Ctrl+F9`).

- Observe the red line. Press a key to close the window.

4. Draw a Vertical Line:

- Add another line:

```
// Task 1.2: Draw a vertical green line  
setcolor(GREEN);  
line(150, 50, 150, 250);
```

- Save, Compile, and Run. Observe both lines.

5. Draw a Rectangle:

- Add the following:

```
// Task 1.3: Draw a blue rectangle  
setcolor(BLUE);  
rectangle(300, 50, 500, 250); // rectangle(left, top, right,  
bottom)
```

- Save, Compile, and Run. Observe all drawn shapes.

6. Experiment 1.4: Draw your Initials with Lines.

- Using setcolor() and line(), try to draw your initials (e.g., if your initials are "J.S.", draw a 'J' and an 'S'). Choose different colors for each letter.
- *Self-check:* Are your initials clearly visible? Did you use setcolor() before each new shape?

Task 2: Circles, Ellipses, and Filling Shapes

Now let's explore drawing curves and filling areas.

1. Add a Circle:

- Add the following code (after your initials from Task 1, or just remove them if you want a cleaner screen for this task):

```
// Task 2.1: Draw a yellow circle  
  
setcolor(YELLOW);  
  
circle(150, 350, 50); // circle(centerX, centerY, radius)
```

- Save, Compile, and Run.

2. Add an Ellipse:

- Add:

```
// Task 2.2: Draw a magenta ellipse  
  
setcolor(MAGENTA);  
  
ellipse(350, 350, 0, 360, 100, 50); // ellipse(centerX, centerY,  
startAngle, endAngle, xRadius, yRadius)  
  
// 0, 360 draws a full ellipse
```

- Save, Compile, and Run.

3. Fill a Rectangle with Color:

- graphics.h allows you to fill closed shapes.
- First, define a fill style:

```
// Task 2.3: Fill a rectangle  
  
setfillstyle(SOLID_FILL, CYAN); // Set fill style to solid,  
color Cyan  
  
bar(500, 300, 600, 400); // bar(left, top, right, bottom) -  
draws and fills a rectangle
```

Note: bar() draws a filled rectangle. For other shapes, you often draw the outline first (rectangle, circle) and then use floodfill() with a specific point inside the shape. Let's try floodfill.

Experiment 2.4: Draw a circle and then fill it.

- Remove the bar() from the previous step.
- Draw a white circle first: setcolor(WHITE); circle(250, 450, 40);

- Then fill it: `setfillstyle(SOLID_FILL, GREEN); floodfill(250, 450, WHITE);`
 - `floodfill(x, y, border_color)`: Fills from (x,y) until it hits `border_color`. (250, 450) is inside the circle, `WHITE` is the color of the circle's border.
- Save, Compile, and Run.

Task 3: Text and Simple Animation Concepts (35 minutes)

Graphics isn't just shapes; it's also about displaying text and creating movement.

1. Display Text:

- Add:

```
// Task 3.1: Display some text

setcolor(WHITE);

outtextxy(10, 10, "Hello, Graphics!"); // outtextxy(x, y, "text
string")
```

- Save, Compile, and Run. The text should appear in the top-left.

2. Change Text Style:

- Try changing the font and size:

```
// Task 3.2: Change text style and position

settextstyle(GOTHIC_FONT, HORIZ_DIR, 4); // Font, Direction
(horizontal), Size (1-4)

setcolor(YELLOW);

outtextxy(100, 50, "Bigger Text!");
```

- Save, Compile, and Run. Experiment with different font styles (e.g., `TRIPLEX_FONT`, `SMALL_FONT`, `SANSABOLD_FONT`) and sizes. Remember to reset the style if you want subsequent text to be normal.

3. Basic Animation - Clearing and Redrawing (Conceptual & Practical):

- To make something appear to move, you typically:
 1. Draw it.
 2. Wait a short time.

3. Clear the screen (or just the old drawing).
 4. Redraw it in a new position.
- graphics.h has a delay() function for waiting.
 - **Remove all previous drawing code** for this task, so you have a clean screen.
 - Add the following:

```
// Task 3.3: Simple moving circle animation
int x_pos = 50;
int y_pos = 200;
int radius = 20;

for (int i = 0; i < 100; i++) // Loop 100 times
{
    setcolor(CYAN);
    circle(x_pos, y_pos, radius); // Draw the circle

    delay(50); // Wait for 50 milliseconds

    setcolor(BLACK); // Set color to background color (usually
black)
    circle(x_pos, y_pos, radius); // "Erase" the old circle by
drawing it in black

    x_pos += 2; // Move the circle 2 pixels to the right
    if (x_pos > getmaxx() - radius) { // Check if it hits the
right edge
        x_pos = 50; // Reset to left
    }
}
// This loop will run, and then the getch() at the end will
pause it.
```

- Save, Compile, and Run. You should see a circle moving across the screen.
- **Note:** getmaxx() and getmaxy() return the maximum X and Y coordinates of the screen.

Conclusion:

- What are the advantages of using graphics.h for learning basic drawing concepts?
- How would you go about drawing a smiley face using the functions you've learned? List the functions you would use.

Troubleshooting Tips:

- **"The graphics window doesn't appear or closes immediately!"**
 - Ensure initgraph is called correctly.
 - Make sure getch() is present *after* your drawing commands and *before* closegraph().
 - Check the initgraph path (the third argument) carefully if using Turbo C/DOSBox.
 - Check the console window for printf messages from graphresult().
- **"My shapes are not the right color!"**
 - Did you call setcolor() *before* drawing the shape?
- **"My text is too small/big/wrong font!"**
 - Remember to use settextstyle() before outtextxy().
- **"My animation is too fast/slow!"**
 - Adjust the number in delay() (larger number = slower animation).